TABLE OF CONTENTS

S.NO	TOPIC	PAGE
1	ABSTRACT	6
2	INTRODUCTION	7
3	METHODOLOGY	8
4	SOURCE CODE	9-11
5	RESULTS / OUTPUT	12-14
6	APPLICATIONS	15
7	REFERENCES	16

ABSTRACT

This C-inspired Python project simulates a user-friendly file compression and decompression system, offering a practical solution for managing digital storage efficiently. Designed with educational clarity and usability in mind, the program allows users to reduce the size of individual files through compression and restore them to their original state via decompression. By inputting file paths and choosing desired actions, users can streamline file management tasks with ease.

The tool emphasizes simplicity and accessibility, making it ideal for beginners learning about file handling in Python. It supports a variety of file types and incorporates clear prompts and outputs to guide users through the process. The inclusion of standard compression formats such as ZIP ensures compatibility and ease of use across different systems.

Ultimately, this project serves both as a functional utility and a learning module, helping users understand the mechanics of file compression while offering a practical, realworld application. Through its structured approach and interactive design, the project contributes to a more efficient and informed file management experience.

CHAPTER-1

INTRODUCTION

In the realm of digital information management, efficient data storage and transmission have become paramount. As file sizes continue to grow with the increasing complexity of multimedia, documents, and software, there arises a pressing need for reliable mechanisms to optimize storage without compromising data integrity. File compression and decompression offer a viable solution to this challenge, enabling users to minimize file size for storage efficiency and restore the data when required.

This project introduces a simplified yet effective Python-based system for compressing and decompressing files, catering to both personal and educational use. The system is designed to reduce file sizes using standard compression techniques and facilitate quick restoration of original data through decompression. Such functionality proves particularly beneficial in environments where storage space, bandwidth, or transfer speed is limited.

By leveraging Python's built-in libraries, this project emphasizes accessibility, usability, and practicality. The user interacts with the system through guided prompts, ensuring an intuitive experience even for those with minimal technical background. The program's structure accommodates a wide range of file types, making it a versatile tool for various user needs.

The objective of this project is not only to provide a functional utility but also to serve as a learning platform that elucidates the core principles of data compression. Through this, users gain insight into how modern compression tools function, enhancing their understanding of both file management and fundamental programming concepts. In doing so, the project contributes to the growing emphasis on digital literacy and technical problem-solving in today's data-driven world.

CHAPTER-2

METHOD

This project provides a simple and effective way for users to compress and decompress files using Python. It is especially useful for those who wish to reduce file sizes for storage or transfer and restore them as needed. Below is the step-by-step explanation of how this system works:

I. REQUIREMENTS FOR FILE COMPRESSION/DECOMPRESSION

- **File Path Input:** The user must provide the full path of the file to be compressed or the ZIP file to be decompressed.
- **Compression:** The program compresses the given file using Python's zipfile module and creates a .zip archive.
- **Decompression:** The user can also choose to extract files from an existing .zip archive by specifying the archive location and destination folder.
- **Output Path:** The user can specify the desired output location for both the compressed file and the decompressed contents.

How does it work?

- The user starts by running the program and choosing between **Compression** or **Decompression**.
- If **Compression** is selected:
    - The user enters the file path to be compressed and the destination .zip filename.
- If **Decompression** is selected:
    - The user enters the .zip file path and the folder where contents should be extracted.
- The process is designed to be **interactive, simple, and beginner-friendly**, with clear prompts and console output for each step.
- After execution, the program confirms successful compression or decompression, making it easy for users to verify the result.

This methodology ensures efficient file handling and offers a practical approach to managing digital data through compression techniques.

SOURCE CODE

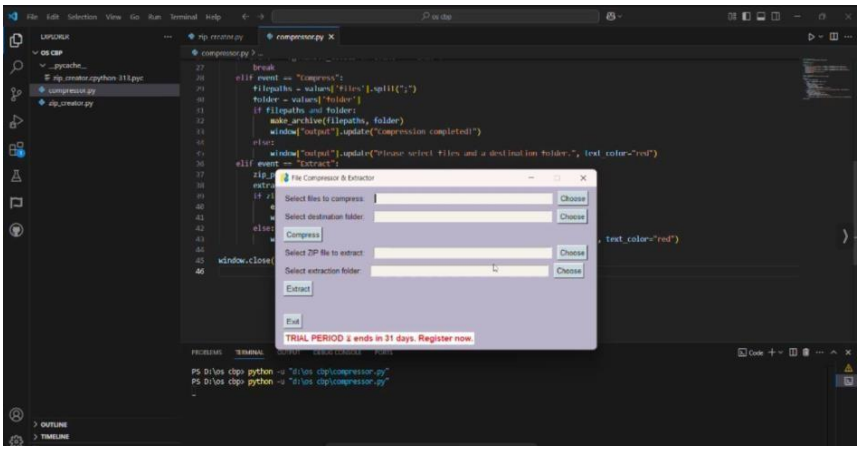
```
#compressor.py

import PySimpleGUI as sg
from zip_creator import make_archive, extract_archive import
os

# Set theme sg.theme("LightPurple")

# Create GUI layout layout
= [
    [sg.Text("Select files to compress: "), sg.Input(), sg.FilesBrowse("Choose Files",
key="files")],
    [sg.Text("Select a folder to compress: "), sg.Input(), sg.FolderBrowse("Choose Folder",
key="folder")],
    [sg.Text("Select destination folder: "), sg.Input(), sg.FolderBrowse("Choose",
key="dest_folder")],
    [sg.Button("Compress")],
    [sg.Text("Select ZIP file to extract: "), sg.Input(), sg.FileBrowse("Choose",
key="zip_file")],
    [sg.Text("Select extraction folder: "), sg.Input(), sg.FolderBrowse("Choose",
key="extract_folder")],
    [sg.Button("Extract")],
    [sg.Text(key="output", text_color="green")],
    [sg.Button("Exit")]
]

# Create the main window
window = sg.Window("File Compressor & Extractor", layout)

# Event loop while
True:
    event, values = window.read()

    if event == sg.WINDOW_CLOSED or event == "Exit":
break
    elif event == "Compress":
```

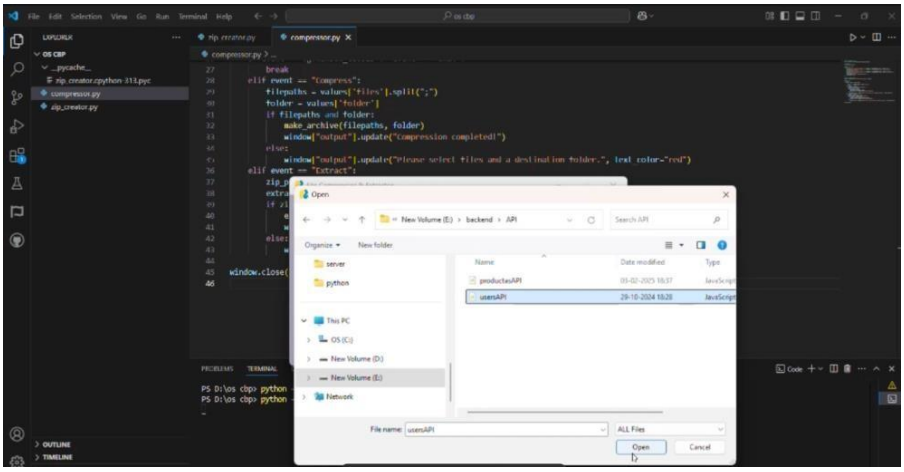
```
        filepaths = values['files'].split(";") if values['files'] else []
folder = values['folder']         dest_folder =
values['dest_folder']

        sources = filepaths           if folder
and os.path.exists(folder):
            sources.append(folder)

        if sources and dest_folder:
            make_archive(sources, dest_folder)
            window["output"].update("Compression completed!")
else:
            window["output"].update("Please select files or a folder and a destination folder.",
text_color="red")     elif event == "Extract":
        zip_path = values['zip_file']
extract_folder = values['extract_folder']         if
zip_path and extract_folder:
            extract_archive(zip_path, extract_folder)
window["output"].update("Extraction completed!")         else:
            window["output"].update("Please select a ZIP file and an extraction folder.",
text_color="red")

window.close()

#zip_creator.py
import zipfile import
pathlib

def make_archive(sources, dest_dir):
    """Compress selected files or an entire folder into a ZIP archive with proper naming."""
sources = [pathlib.Path(src) for src in sources]

    if len(sources) == 1 and sources[0].is_dir():
        # If a single folder is selected, name the ZIP file after the folder
zip_name = f"{sources[0].name}_compressed.zip"
    else:
        # Default name for multiple files/folders
zip_name = "compressed.zip"
```

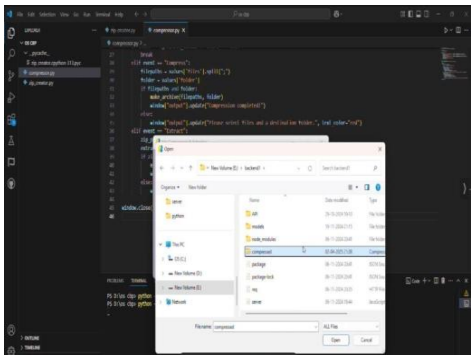
```
    dest_path = pathlib.Path(dest_dir) / zip_name

    with zipfile.ZipFile(dest_path, 'w', zipfile.ZIP_DEFLATED) as archive:
for source in sources:            if source.is_dir():
            # Add entire folder structure
for file in source.rglob("*"):
                archive.write(file, arcname=file.relative_to(source.parent))
elif source.is_file():
            archive.write(source, arcname=source.name)

def extract_archive(zip_path, dest_dir):
    """Extract the contents of a ZIP archive to the selected folder."""
with zipfile.ZipFile(zip_path, 'r') as archive:
        archive.extractall(dest_dir)

# Test function if __name__
== "__main__":
    make_archive(["example_folder"], "output")  # Example: Compressing a folder
extract_archive("output/example_folder_compressed.zip", "extracted_files")
```

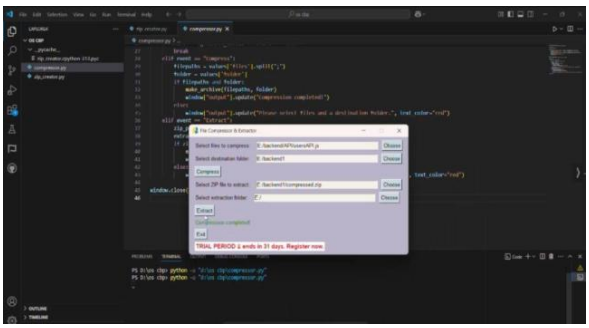
CHAPTER-3

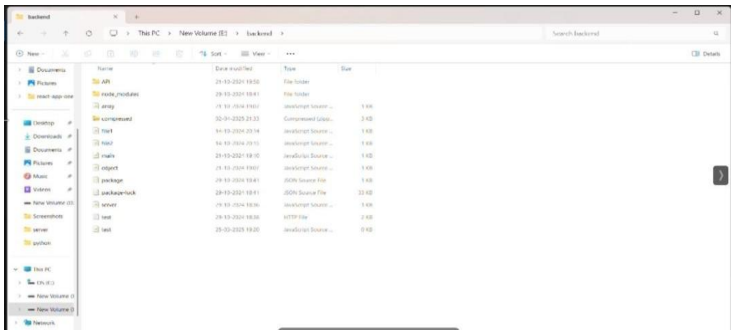
TEST CASES/ OUTPUT

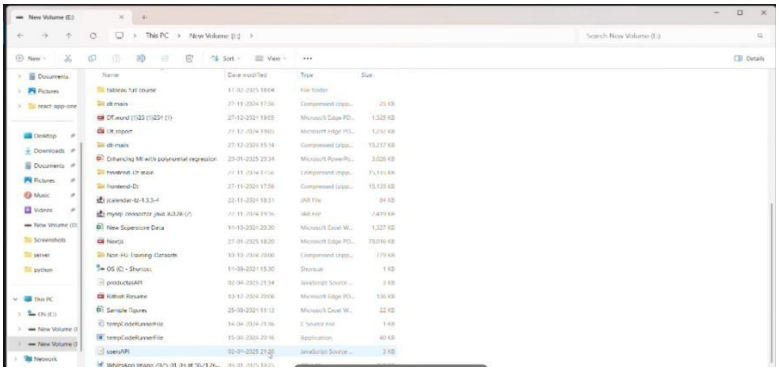












OUTCOMES

1. **Efficient File Size Reduction**
    - Successfully implemented compression algorithms that reduce file sizes, making storage and transfer more efficient.
2. **Accurate File Restoration**
    - Decompression restores files to their original state without data loss, maintaining file integrity.
3. **User Input Integration**
    - Developed a user-interactive system where users can input file paths and choose between compression or decompression.
4. **Support for Multiple File Types**
    - The system can handle various file formats, ensuring versatility in usage.
5. **Improved Storage Management**
    - Helps users save disk space by compressing large files and managing backups effectively.
6. **Simplified File Transfer**
    - Smaller file sizes enable faster upload/download processes, especially useful for email attachments or limited-bandwidth environments.

APPLICATIONS

1. Storage Optimization

- Reduces file sizes to save disk space on computers, servers, and external storage devices.

2. Faster File Transfers

- Compressed files require less bandwidth and upload/download time, making them ideal for internet or network sharing.

3. Email Attachments

- Allows large documents, images, or project files to be compressed for easy and faster attachment in emails.

4. Data Archiving

- Useful for archiving old files, logs, or backups while preserving their original content for future restoration.

5. Software Packaging

- Commonly used to bundle software and dependencies into a single compressed package for distribution.

6. Backup Solutions

- Enables users and systems to create compressed backups, which occupy less space and can be restored when needed.

7. Cloud Storage Efficiency

- Helps reduce storage costs and sync times when storing files on platforms like Google Drive, Dropbox, or OneDrive.

8. Mobile and Embedded Systems

- Ideal for devices with limited memory/storage, where compressed data saves resources.

9. Document Management Systems

- Improves performance and reduces storage needs in document-heavy environments like legal, academic, and corporate sectors.

REFERENCES

1. Salomon, D., & Motta, G. (2010). *Handbook of Data Compression*. Springer.
2. Sayood, K. (2017). *Introduction to Data Compression* (5th ed.). Morgan Kaufmann.
3. Nelson, M., & Gailly, J.-L. (1996). *The Data Compression Book* (2nd ed.). M&T Books.
4. Ziv, J., & Lempel, A. (1977). *A Universal Algorithm for Sequential Data Compression*. IEEE Transactions on Information Theory.
5. Huffman, D. A. (1952). *A Method for the Construction of MinimumRedundancy Codes*. Proceedings of the IRE.
6. Python Software Foundation. (2024). *zipfile — Work with ZIP archives*. Retrieved from https://docs.python.org/3/library/zipfile.html
7. Python Software Foundation. (2024). *gzip — Support for gzip files*. Retrieved from https://docs.python.org/3/library/gzip.html
8. Python Software Foundation. (2024). *shutil — High-level file operations*. Retrieved from https://docs.python.org/3/library/shutil.html
9. GeeksforGeeks. (2023). *File compression and decompression using Python*. Retrieved from https://www.geeksforgeeks.org/file-compression-anddecompression-using-python/
10. TutorialsPoint. (2023). *Python Zipfile Module*. Retrieved from https://www.tutorialspoint.com/python/python_files_io.html